Studio Nx

S1-08 아코디언·피아노롤 UI — 함수 설명 합본 (JS 219~225)

2026-10-17

목차

1. ai-notes/javascript/S1-08 - 아코디언·피아노롤/219_sequencer-arrange_ruler_seek.md
2. ai-notes/javascript/S1-08 - 아코디언·피아노롤/220_sequencer-roll_shift_click.md
3. ai-notes/javascript/S1-08 - 아코디언·피아노롤/221_sequencer-arrange_track_select_group.md
4. ai-notes/javascript/S1-08 - 아코디언·피아노롤/222_sequencer-ui_track_menu.md
5. ai-notes/javascript/S1-08 - 아코디언·피아노롤/223_sequencer-arrange_open_panels.md
6. ai-notes/javascript/S1-08 - 아코디언·피아노롤/224_sequencer-roll_emit_notes.md
7. ai-notes/javascript/S1-08 - 아코디언·피아노롤/225_sequencer_clip_preview.md

JavaScript 219 — 마디 눈금 클릭으로 플레이헤드 (`rulerBar` · `initPan`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 2번 (마디 눈금 클릭 → 플레이헤드 이동)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	218 패턴 라우터(노트·수정) · 220 겹쳐 찍기

1. 하는 일

편곡 화면 위쪽 **마디 눈금**(`.sq-ruler__bars`) 줄을 클릭하면 그 자리로 플레이헤드가 옮겨집니다. 끌지 않고 그냥 누른 경우만 반응하고, 끌면 기존의 좌우 이동(팬)이 됩니다. `Alt` 를 누르면 격자에 붙이지 않고 Free 로 놓습니다.

```
눈금 줄 pointerdown → pan.ruler 에 그 줄을 기억
pointerup (안 끌었을 때) → setPlayheadBar(clamp(rulerBar(...), 0, totalBars))
```

2. 언제 쓰나

쓰는 곳	
편곡 화면	재생을 멈춘 채 원하는 마디로 바로 점프해 확인할 때
사용자 요구 2번	"마디 눈금 클릭하면 재생 위치가 그쪽으로 이동"

3. 코드

```
/** 마디 눈금에서 클릭한 x → 마디 (눈금 줄 왼쪽 = 0마디, Alt 면 스냅 없이). 클릭 = 플레이헤드 옮기기 */
function rulerBar(barEl, clientX, free, lanes) {
  return snap((clientX - barEl.getBoundingClientRect().left) / barWidth(lanes), free);
}
```

```
function initPan({ scroll, lanes }) {
  const PAN_AREA = '.sq-ruler__bars, .sq-markers__track, .sq-track__lane, .sq-fill__lane';
  let pan = null;

  scroll.addEventListener('pointerdown', (e) => {
    const middle = e.button === 1;
    if (!middle && e.button !== 0) return;
    if (!middle && (!e.target.closest(PAN_AREA) || e.target.closest('button, .sq-auto, .sq-loop'))) retu
rn;
    // …도구별 예외(영역 선택·툴 동작)는 그대로 통과…
    pan = {
      x: e.clientX, left: scroll.scrollLeft, id: e.pointerId, moved: false,
      clear: !middle && !e.ctrlKey && !e.metaKey && !!e.target.closest('.sq-track__lane, .sq-fill__lan
```

```
e'),
      // 마디 눈금을 그냥 클릭(끌지 않음) → 플레이헤드를 그 자리로 (FR-2331)
      ruler: !middle ? e.target.closest('.sq-ruler__bars') : null,
      draw: !middle && currentTool === 'draw' ? { lane: e.target.closest('.sq-track__lane'), x: e.client
X, free: e.altKey } : null,
    };
    if (middle) e.preventDefault();
  });
  // …pointermove 는 scroll.scrollLeft 를 움직이는 팬…
  const end = (e) => {
    if (!pan || e.pointerId !== pan.id) return;
    if (pan.moved) scroll.releasePointerCapture(e.pointerId);
    else if (pan.ruler) setPlayheadBar(clamp(rulerBar(pan.ruler, e.clientX, e.altKey, lanes), 0, totalBa
rs(lanes)));
    else if (pan.draw?.lane && drawClipAt(pan.draw.lane, pan.draw.x, pan.draw.free)) { /* 놓음 */ }
    else if (pan.clear) clearSelection(scroll);
    scroll.classList.remove('is-panning');
    pan = null;
  };
  scroll.addEventListener('pointerup', end);
  scroll.addEventListener('pointercancel', end);
}
```

문법 풀이

식	뜻
`barEl.getBoundingClientRect().left`	눈금 줄의 화면상 왼쪽 좌표입니다
`(clientX - left) / barWidth(lanes)`	클릭한 x 를 마디 수로 바꿉니다
`snap(값, free)`	격자에 붙입니다. `free`(Alt)면 그대로 둡니다
`pan.ruler`	누른 대상이 눈금 줄이면 그 줄을 기억합니다 (아니면 `null`)
`!pan.moved`	끌지 않은(그냥 누른) 경우만 플레이헤드를 옮깁니다
`clamp(v, 0, totalBars(lanes))`	0 ~ 전체 마디 사이로 값을 가둡니다

4. 왜 이렇게

선택	이유
끌면 팬, 안 끌면 점프	같은 줄에서 좌우 이동도 자주 씁니다. 끌었는지로 둘을 가릅니다
`pointerup` 에서 판단	누르는 순간 움직이면 스크롤하려다 실수로 재생 위치가 바뀝니다
`pan.ruler` 를 미리 기억	`pointerup` 의 대상(`e.target`)은 누른 요소가 아닐 수 있어, 누를 때 기억해 둡니다

5. 시험

헤드리스 브라우저에서 눈금 4마디 지점을 클릭 → 플레이헤드 `--at` 값이 처음 값에서 `4` 로 바뀌는 것을 확인했습니다(콘솔 오류 0건).

JavaScript 220 — 겹쳐 찍기 (`initNoteEdit` 의 Shift+클릭)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-roll.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 4번 (노트 위에 겹쳐 찍기)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	219 마디 눈금 클릭 · 221 트랙 선택·그룹

1. 하는 일

피아노롤에서 **이미 있는 노트 위**를 `Shift` 를 누른 채 클릭하면, 그 노트를 옮기지 않고 **같은 음높이·시작 위치에 새 노트를 하나 더** 찍습니다(겹침 허용, FR-2317). 빈 곳 클릭은 늘 새로 찍습니다.

```
pointerdown (좌측 버튼)
├ 노트 위가 아니거나 Shift 인 경우 → 새 노트 (Shift 면 아래 노트의 pitch·start 를 물려받음)
│   └ 마지막 길이(R.lastLength)·세기 100 으로 만들고 붙임 → onChange({type:'add'})
└ 그 외 → 선택/이동/크기 조절 (기존 동작 그대로)
```

2. 언제 쓰나

쓰는 곳	
피아노롤	같은 음을 여러 번 겹쳐 두고 세기를 달리하거나, 코드를 쌓을 때
사용자 요구 4번	"노트를 겹쳐 찍을 수 있게 해 달라"

3. 코드

```
notes.addEventListener('pointerdown', (e) => {
  const target = e.target.closest('.sq-note:not(.is-ghost)');
  const multi = e.ctrlKey || e.metaKey;
  // …오른쪽 클릭 = 지우기…
  if (e.button !== 0) return;
  let note = target;
  let added = false;
  // 빈 곳 클릭 = 찍기 · Shift+클릭(노트 위) = 그 노트에 겹쳐 새로 찍기 (FR-2317 노트 겹침 허용)
  if (!note || e.shiftKey) {
    let pitch = null;
    let start = null;
    if (note) {                                  // 노트 위 + Shift → 그 노트 자리에 겹쳐 찍기
      pitch = Number(note.dataset.pitch);
      start = clamp(Number(note.dataset.start), 0, R.rollBars - R.lastLength);
    } else {                                     // 빈 곳 → 클릭한 음높이·위치에 새로 찍기
      const { bar, row } = point(e);
      pitch = R.high - row;
      if (pitch < R.low || pitch > R.high) return;
      start = clamp(snapF(bar, e.altKey), 0, R.rollBars - R.lastLength);
    }
```

```
    const n = document.createElement('span');
    n.className = 'sq-note';
    n.dataset.pitch = pitch;
    n.dataset.start = start;
    note = n;
    note.dataset.length = R.lastLength;
    note.dataset.vel = 100;
    notes.append(note);
    placeNote(note);
    renderVelocity();
    added = true;
    select(note, false);
  } else if (!note.classList.contains('is-selected')) {
    select(note, multi);
  }
  // …아래는 끌기(이동) 준비…
}
```

문법 풀이

식	뜻
`e.target.closest('.sq-note:not(.is-ghost)')`	클릭한 곳에서 가장 가까운 노트를 찾습니다(미리보기 노트는 제외)
`!note	
`Number(note.dataset.pitch)`	노트 요소의 `data-pitch` 값을 숫자로 읽습니다
`point(e)`	클릭 좌표를 `{ bar, row }` 로 바꿉니다
`placeNote(note)`	요소를 위치·길이에 맞게 화면에 놓습니다

4. 왜 이렇게

선택	이유
Shift 를 겹침에 배정	같은 자리에서 옮기기 대신 새로 찍는 일은 드뭅니다. 실수로 덮어쓰지 않게 수식 키를 씁니다
겹칠 땐 아래 노트 값을 물려받음	겹쳐 찍는 목적은 보통 같은 음·위치에 다른 세기를 주는 것입니다
`added` 표시	새로 찍은 노트는 곧바로 끌지 않고, 세기 편집용으로만 씁니다

5. 시험

헤드리스 브라우저에서 일반 클릭은 노트 수가 그대로, Shift+클릭은 노트가 1개 늘고 같은 음높이·시작 위치의 겹친 쌍이 생기는 것을 확인했습니다(콘솔 오류 0건).

JavaScript 221 — 트랙 선택과 그룹 (`selectTrack` · `groupSelectedTracks` · `ungroupTracks` · `deleteSelectedTracks` · `addGroup`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 3번 (트랙 선택 + 그룹화, FR-207)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	220 겹쳐 찍기 · 222 트랙 메뉴·단축키

1. 하는 일

트랙 머리의 빈 곳을 클릭해 트랙을 고르고, 고른 트랙들을 그룹으로 묶거나 풉니다. 그룹은 안쪽 트랙을 접었다 펼 수 있고, 그룹을 풀거나 지우면 안쪽 트랙은 다시 밖으로 나옵니다.

함수	하는 일
`selectedTracks()`	지금 선택된 트랙 줄 (MASTER 제외)
`clearTrackSelection()`	트랙 선택 풀기
`selectTrack(track, e)`	그냥 클릭 = 이것만, Ctrl/Cmd = 더하고 빼기, Shift = 기준 트랙에서 여기까지 모두
`createGroupEl()`	`Group N` 그룹 줄 하나를 만듦
`updateGroupCount(group)`	그룹 머리의 `N tracks` 글자를 맞춤
`addGroup(notify)`	빈 그룹 하나를 리턴·MASTER 위에 넣음
`groupSelectedTracks(notify)`	선택 트랙들을 새 그룹으로 묶음 (선택 없으면 빈 그룹)
`ungroupTracks(notify)`	선택 그룹을 풀어 안쪽 트랙을 밖으로 냄
`deleteSelectedTracks(notify)`	선택 트랙을 지움 (그룹이면 안쪽을 밖으로 낸 뒤)

2. 언제 쓰나

쓰는 곳	
트랙 머리 클릭 (2654 `initTrackButtons`)	`selectTrack` 을 부릅니다
오른쪽 클릭 메뉴 (222) · `Ctrl+G`	`groupSelectedTracks`·`ungroupTracks`·`deleteSelectedTracks` 를 부릅니다
ADD › Group track	`addGroup` 을 부릅니다

```
selectTrack(track, e)
├─ Shift + 기준 트랙 있음 → 기준~여기 사이 모두
├─ Ctrl/Cmd              → 그 트랙만 더하거나 빼기 (기준 갱신)
└─ 그냥 클릭              → 이것만 선택 (기준 갱신)
```

3. 코드

```
export function selectTrack(track, e = {}) {
  if (!track || track.classList.contains('sq-track--master')) return;
  const list = trackItems();
  const multi = e.ctrlKey || e.metaKey;
  if (e.shiftKey && trackAnchor && list.includes(trackAnchor)) {
    const a = list.indexOf(trackAnchor);
    const b = list.indexOf(track);
    const lo = Math.min(a, b);
    const hi = Math.max(a, b);
    list.forEach((t, i) => t.classList.toggle('is-selected', i >= lo && i <= hi && !t.classList.contains
('sq-track--master')));
  } else if (multi) {
    track.classList.toggle('is-selected');
    trackAnchor = track;
  } else {
    list.forEach((t) => t.classList.toggle('is-selected', t === track));
    trackAnchor = track;
  }
}

export function groupSelectedTracks(notify = arrangeOnChange) {
  const sel = selectedTracks();
  if (!sel.length) return addGroup(notify);
  const group = createGroupEl();
  sel[0].before(group);
  [...sel].reverse().forEach((t) => {
    const acc = t.nextElementSibling;                       // 펼침 칸은 트랙 바로 뒤에 붙어 있음
    const withAcc = acc && acc.classList.contains('sq-acc') ? acc : null;
    group.after(t);                                         // 역순으로 넣어 원래 차례를 지킴
    if (withAcc) t.after(withAcc);
    t.dataset.parent = group.dataset.trackId;
    t.classList.add('is-child');
  });
  clearTrackSelection();
  group.classList.add('is-selected');
  trackAnchor = group;
  updateGroupCount(group);
  notify({ type: 'group', trackId: group.dataset.trackId, children: sel.map((t) => t.dataset.trackId)
});
  return group;
}
```

문법 풀이

식	뜻
`trackAnchor`	Shift 로 "여기까지" 를 잴 기준 트랙을 담아 둡니다
`list.indexOf(t)`	그 줄이 목록에서 몇 번째인지 구합니다
`classList.toggle('is-selected', 참거짓)`	두 번째 값이 참이면 붙이고 거짓이면 뗍니다
`sel[0].before(group)`	첫 선택 트랙 **앞**에 그룹 줄을 넣습니다
`[...sel].reverse()`	뒤에서부터 옮겨 원래 위→아래 차례를 지킵니다

식	뜻
`dataset.parent`	안쪽 트랙에 부모 그룹 번호를 적어 둡니다
`notify({ type: 'group', ... })`	저장을 위해 바뀐 사실을 알립니다 (서버 연결은 S1-10)

4. 왜 이렇게

선택	이유
MASTER 는 항상 제외	MASTER 는 옮기거나 지울 수 없는 줄입니다. 선택 대상에서 뺍니다
펼침 칸도 함께 옮김	트랙 바로 뒤에 `.sq-acc`(아코디언)가 붙어 있어, 트랙만 옮기면 칸이 떨어집니다
그룹을 풀면 안쪽을 밖으로	안쪽 트랙을 숨긴 채 그룹만 지우면 트랙이 사라진 것처럼 보입니다. 밖으로 낸 뒤 지웁니다
`notify` 로 알림	서버 저장(자동 저장)은 S1-10 이 붙입니다. 지금은 바뀐 사실만 알립니다

5. 시험

헤드리스 브라우저에서 트랙 2개를 골라 `Ctrl+G` → 그룹이 1개 생기고 안쪽 2개가 `data-parent` 로 묶이며 접기/펼치기가 되는 것을 확인했습니다(콘솔 오류 0건).

JavaScript 222 — 트랙 오른쪽 클릭 메뉴와 단축키 (`groupTracks` · `ungroupSelection` · `deleteTracks` · `openTrackMenu` · `initTrackMenu`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-ui.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 3번 (트랙 선택 + 그룹 메뉴, FR-207)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	221 트랙 선택·그룹 · 223 펼침 상태 기억

1. 하는 일

트랙 머리를 오른쪽 클릭하면 작은 메뉴(그룹으로 묶기 · 그룹 풀기 · 트랙 지우기)가 뜹니다. 같은 일을 `Ctrl+G`·`Ctrl+Shift+G`·`Delete` 키로도 할 수 있습니다. 실제 동작은 편곡 함수(221)를 부르고, 결과를 토스트로 알립니다.

함수	하는 일
`groupTracks()`	선택 트랙을 그룹으로 묶고 토스트
`ungroupSelection()`	선택 그룹을 풀고 토스트
`deleteTracks()`	선택 트랙을 지우고 토스트
`openTrackMenu(x, y)`	마우스 자리에 메뉴를 띄우고 항목 처리를 연결
`initTrackMenu()`	트랙 오른쪽 클릭과 단축키를 배선

2. 언제 쓰나

쓰는 곳	
트랙 머리에서 오른쪽 클릭	`openTrackMenu` 로 메뉴를 띄웁니다
`Ctrl+G` · `Ctrl+Shift+G`	`groupTracks` · `ungroupSelection`
Delete 툴바/단축키	`deleteTracks`

3. 코드

```
function groupTracks() {
  const n = selectedTracks().length;
  const g = groupSelectedTracks();
  if (!g) return;
  showToast(n ? `Grouped ${n} track${n === 1 ? '' : 's'}` : 'New group track added');
}

function openTrackMenu(x, y) {
  document.querySelector('.sq-trackmenu')?.remove();
  const n = selectedTracks().length;
  const hasGroup = selectedTracks().some((t) => t.classList.contains('sq-track--group'));
```

```
  const ul = document.createElement('ul');
  ul.className = 'sq-dropdown is-popup sq-trackmenu';
  ul.setAttribute('role', 'menu');
  ul.innerHTML = `
    <li><button type="button" role="menuitem" data-act="group"${n ? '' : ' disabled'}>Group selected tra
cks <kbd>Ctrl+G</kbd></button></li>
    <li><button type="button" role="menuitem" data-act="ungroup"${hasGroup ? '' : ' disabled'}>Ungroup <
kbd></kbd></button></li>
    <li role="separator"><hr></li>
    <li><button type="button" role="menuitem" data-act="delete">Delete track <kbd>Del</kbd></button></li
>`;
  ul.style.position = 'fixed';
  ul.style.left = `${Math.min(x, window.innerWidth - 230)}px`;
  ul.style.top = `${Math.min(y, window.innerHeight - 150)}px`;
  document.body.append(ul);
  const off = (e) => { if (!ul.contains(e.target)) close(); };
  const esc = (e) => { if (e.key === 'Escape') close(); };
  function close() {
    ul.remove();
    document.removeEventListener('pointerdown', off);
    document.removeEventListener('keydown', esc);
  }
  ul.addEventListener('click', (e) => {
    const b = e.target.closest('button[data-act]');
    if (!b) return;
    const act = b.dataset.act;
    close();
    if (act === 'group') groupTracks();
    else if (act === 'ungroup') ungroupSelection();
    else if (act === 'delete') deleteTracks();
  });
  setTimeout(() => { document.addEventListener('pointerdown', off); document.addEventListener('keydown',
esc); });
}

function initTrackMenu() {
  const lanes = document.getElementById('lanes');
  if (!lanes) return;
  // 트랙 머리에서 오른쪽 클릭 = 그 트랙 선택 후 메뉴
  lanes.addEventListener('contextmenu', (e) => {
    const head = e.target.closest('.sq-track__head');
    const track = head?.closest('.sq-track');
    if (!track || e.target.closest('button')) return;
    e.preventDefault();
    if (!track.classList.contains('is-selected')) selectTrack(track);
    openTrackMenu(e.clientX, e.clientY);
  });
  // Ctrl+G = 선택 트랙 그룹화 · Ctrl+Shift+G = 그룹 풀기
  document.addEventListener('keydown', (e) => {
    if (!(e.ctrlKey || e.metaKey) || e.key.toLowerCase() !== 'g') return;
    if (e.target.closest?.('input, textarea, select, [contenteditable]') || document.querySelector('dial
og[open]')) return;
    e.preventDefault();
    if (e.shiftKey) ungroupSelection(); else groupTracks();
  });
}
```

문법 풀이

식	뜻
`document.querySelector('.sq-trackmenu')?.remove()`	이미 떠 있던 메뉴가 있으면 지웁니다 (`?.` 는 없으면 건너뜀)
`ul.setAttribute('role', 'menu')`	보조기기에 이 상자가 메뉴임을 알립니다
`${n ? '' : ' disabled'}`	선택이 없으면 "그룹으로" 항목을 비활성으로 둡니다
`ul.style.left = Math.min(x, innerWidth - 230)`	메뉴가 화면 밖으로 나가지 않게 자리를 좁힙니다
`setTimeout(...)`	메뉴를 여는 그 클릭에 곧바로 닫히지 않게 잠깐 뒤에 바깥 클릭 감지를 붙입니다
`e.preventDefault()`	오른쪽 클릭 기본 메뉴(브라우저) 가 뜨지 않게 합니다

4. 왜 이렇게

선택	이유
동작은 221 을 그대로 부름	그룹·삭제 로직은 편곡 파일 한 곳(221)에만 둡니다. 메뉴는 그 함수를 부르기만 합니다
항목을 못 쓰는 동안 `disabled`	선택이 없는데 "그룹으로 묶기"를 누르면 빈 그룹이 생겨 헷갈립니다
화면 밖으로 안 나가게	창 가장자리에서 오른쪽 클릭해도 메뉴가 잘리지 않습니다

5. 시험

헤드리스 브라우저에서 트랙 머리 오른쪽 클릭 → 메뉴 3항목이 뜨고, 그룹 항목을 누르면 그룹이 생기는 것을 확인했습니다(콘솔 오류 0건).

JavaScript 223 — 펼침 상태 기억 (`saveOpenPanels` · `restoreOpenPanels`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-08 아코디언·피아노롤: 트랙 아코디언 펼침 상태 기억 (FR-261 의 V 토글에 더한 사용자 요구)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	222 트랙 메뉴·단축키 · 224 노트 변화 알림

1. 하는 일

V 버튼으로 트랙을 펼칠 때마다 **지금 펼쳐 둔 트랙 번호**를 `localStorage` 에 담아 둡니다. 화면을 다시 열면 담아 둔 트랙들을 그대로 다시 펼칩니다. FR-261 의 원래 규칙(기본은 하나만 열림, Shift 로 여러 개, 펼친 만큼 아래 트랙을 밀기)은 `toggleTrackPanel` 이 이미 지키고 있고, 이 함수는 그 상태를 **기억**하는 부분을 맡습니다.

```
V 클릭 → toggleTrackPanel (열기/닫기 · 하나만/여러 개)
          └ saveOpenPanels()            ← 펼쳐진 트랙 번호를 담음
화면 다시 열기 → restoreOpenPanels()     ← 담아 둔 트랙을 다시 펼침
```

2. 언제 쓰나

쓰는 곳	
`toggleTrackPanel` (2367)	상태가 바뀐 뒤 `saveOpenPanels` 를 부릅니다
`sequencer.js` 시작부	트랙이 그려진 뒤 `restoreOpenPanels` 를 부릅니다

3. 코드

```
/** V — 그 트랙 펼침 열고 닫기 (기본은 하나만, Shift 면 여러 개) */
export function toggleTrackPanel(track, keepOthers = false) {
  const btn = track.querySelector('.sq-expand');
  const acc = ensureAccordion(track);
  if (!btn || !acc) return;
  const open = acc.hidden;
  if (open && !keepOthers) {
    document.querySelectorAll('.sq-acc:not([hidden])').forEach((a) => {
      a.hidden = true;
      const t = a.previousElementSibling;
      t?.classList.remove('is-open');
      t?.querySelector('.sq-expand')?.setAttribute('aria-expanded', 'false');
    });
  }
  acc.hidden = !open;
  btn.setAttribute('aria-expanded', String(open));
  track.classList.toggle('is-open', open);
  saveOpenPanels();
}
```

```
const PANEL_KEY = 'sq-open-tracks';

function saveOpenPanels() {
  try {
    const ids = [...document.querySelectorAll('#tracks > .sq-track.is-open')]
      .map((t) => t.dataset.trackId)
      .filter(Boolean);
    localStorage.setItem(PANEL_KEY, JSON.stringify(ids));
  } catch { /* 저장 못 하는 환경은 그냥 넘어감 */ }
}

export function restoreOpenPanels() {
  let ids = [];
  try { ids = JSON.parse(localStorage.getItem(PANEL_KEY) || '[]'); } catch { ids = []; }
  if (!Array.isArray(ids) || !ids.length) return;
  for (const id of ids) {
    const track = document.querySelector(`#tracks > .sq-track[data-track-id="${id}"]`);
    if (track && !track.classList.contains('is-open')) toggleTrackPanel(track, true);
  }
}
```

문법 풀이

식	뜻
`PANEL_KEY`	localStorage 에 담을 때 쓰는 열쇠 이름(`sq-open-tracks`)입니다
`querySelectorAll('#tracks > .sq-track.is-open')`	지금 펼쳐진 트랙 줄만 골라냅니다
`.map((t) => t.dataset.trackId)`	각 줄의 `data-track-id` 를 모읍니다
`.filter(Boolean)`	번호가 없는 줄을 걸러 냅니다
`JSON.stringify(ids)`	번호 배열을 글자로 바꿔 담습니다
`JSON.parse(...	
`toggleTrackPanel(track, true)`	`true` (keepOthers) 로 여러 트랙을 차례로 펼칩니다
`try { … } catch { … }`	저장을 못 하는 환경도 있으니 실패해도 앱이 멈추지 않게 합니다

4. 왜 이렇게

선택	이유
트랙 번호로 담음	순서가 바뀌어도 번호가 같으면 다시 펼 수 있습니다
`keepOthers=true` 로 복원	펼친 트랙이 여럿일 수 있어 하나씩 순서대로 열어야 합니다
저장 실패를 삼킴	시크릿 모드처럼 저장이 막힌 곳에서도 나머지 기능은 그대로 돌아야 합니다

5. 시험

헤드리스 브라우저에서 트랙 1과 3을 펼친 뒤 새로 고침 → 두 트랙이 그대로 펼쳐지고(`is-open` 2개, 아코디언 2개 표시) 저장 값이 `["1","3"]` 인 것을 확인했습니다. Shift 없이 다른 트랙을 펼치면 그 하나만 남는 것도 확인했습니다(콘솔 오류 0건).

JavaScript 224 — 노트 변화 알림 (`rollNotes` · `emitRollNotes` · `openRoll` 의 clip)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-roll.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 1번 (MIDI 클립 미리보기) 의 보내는 쪽
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	223 펼침 상태 기억 · 225 클립 미리보기 그리기

1. 하는 일

피아노롤에서 노트를 찍거나 옮기고 지울 때마다, 지금 노트 전부를 **저장용 모양(tick 단위)** 으로 모아 `sq:roll-notes` 라는 신호로 알립니다. 이 신호를 `sequencer.js` (225) 가 듣고 그 MIDI 클립의 미리보기를 다시 그립니다. 어느 클립의 노트인지는 피아노롤을 열 때 받은 `R.clip` 으로 압니다.

```
노트 편집(찍기·옮기기·지우기)
└ R.onChange(change)
   ├ 사용자 콜백(userChange)      ← 나중에 서버 저장(215)을 붙일 자리
   └ emitRollNotes()
      └ document 에 'sq:roll-notes' 보냄  (detail: { clip, notes, bars })
```

2. 언제 쓰나

쓰는 곳	
`initNoteEdit` (220) · Delete 동작	노트를 바꾼 뒤 `R.onChange` 를 부릅니다
`openRoll(clip)`	열 때 `R.clip` 을 그 클립으로 정합니다
`sequencer.js` (225)	`'sq:roll-notes'` 를 듣고 미리보기를 다시 그립니다

3. 코드

initRoll 안 — 사용자 콜백 뒤에 알림을 붙임

```
const userChange = onChange;
R.onChange = (change) => {
  if (typeof userChange === 'function') userChange(change);
  emitRollNotes();
};
```

openRoll — 어느 클립인지 기억

```
export function openRoll(clip) {
  const bottom = document.getElementById('bottom');
  bottom.hidden = false;
```

```
  bottom.querySelectorAll('.sq-panel').forEach((p) => { p.hidden = p.id !== 'panel-roll'; });
  R.clip = clip || null;
  // …제목·트랙 이름 채우기…
}
```

노트 모으기와 알림

```
/** 지금 피아노롤의 노트 전부를 저장용 모양(tick 단위)으로 모은다 (요구사항 1, 2026-10-17) */
function rollNotes() {
  return [...R.notes.querySelectorAll('.sq-note:not(.is-ghost)')].map((n) => ({
    pitch: Number(n.dataset.pitch),
    start_tick: Math.round(Number(n.dataset.start) * TICKS_PER_BAR),
    length_tick: Math.round(Number(n.dataset.length) * TICKS_PER_BAR),
    velocity: Number(n.dataset.vel) || 100,
  }));
}

/** 노트가 바뀔 때마다 알린다 — sequencer.js 가 듣고 그 MIDI 클립의 미리보기를 다시 그린다 */
function emitRollNotes() {
  if (!R || !R.clip) return;
  document.dispatchEvent(new CustomEvent('sq:roll-notes', {
    detail: { clip: R.clip, notes: rollNotes(), bars: R.patternBars },
  }));
}
```

문법 풀이

식	뜻
`R.clip = clip	
`R.notes.querySelectorAll('.sq-note:not(.is-ghost)')`	실제 노트만 고릅니다(끄는 중 미리보기 노트 제외)
`Number(n.dataset.start) * TICKS_PER_BAR`	화면의 마디 단위를 tick 으로 바꿉니다 (DB·API 와 같은 단위)
`new CustomEvent('sq:roll-notes', { detail: … })`	내려보낼 값(`detail`)을 담은 알림 이벤트를 만듭니다
`document.dispatchEvent(...)`	문서 전체에 알립니다. 듣는 쪽(225)이 받습니다
`if (!R || !R.clip) return`	아직 안 열렸거나 클립이 없으면 알리지 않습니다

4. 왜 이렇게

선택	이유
tick 단위로 바꿔 보냄	DB(`midi_notes`)와 API(213)가 tick 을 씁니다. 화면 단위(마디)를 그대로 보내면 저장할 때 어긋납니다
사용자 콜백을 따로 보존	나중에 서버 저장(215)을 붙일 때 이 자리에 넣습니다. 알림은 그와 별개로 항상 나갑니다
이벤트로 알림	피아노롤과 편곡 화면이 서로를 직접 부르지 않아도 됩니다. 파일 사이 결합을 줄입니다
`is-ghost` 제외	끄는 동안의 미리보기 노트는 진짜 노트가 아닙니다

5. 시험

헤드리스 브라우저에서 피아노롤 노트를 편집(`deleteSelectedNotes`)했을 때 `'sq:roll-notes'` 가 그 클립과 노트 목록·마디 수(`bars: 4`)를 담아 나가고, 그 결과 클립 미리보기가 다시 그려지는 것을 확인했습니다(콘솔 오류 0

건).

JavaScript 225 — MIDI 클립 미리보기 (`setClipNotes` · `drawClipPreview` · `initClipPreviews`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-08 아코디언·피아노롤: 사용자 요구 1번 (MIDI 패턴 멜로디·음정으로 클립 미리보기)
작성	Claude (AI), 2026-10-17, 사용자 검토
이전 · 다음	224 노트 변화 알림 · 226 (예정)

1. 하는 일

MIDI 클립 안의 캔버스(`.sq-clip__preview`)에 **노트를 실제 음높이와 길이대로 작은 막대**로 그립니다. 그래서 클립만 봐도 어떤 멜로디인지 알 수 있습니다. 피아노롤에서 노트가 바뀔 때마다(224 의 `'sq:roll-notes'`) 그 클립을 다시 그립니다. 오디오 클립은 편곡의 `drawWaveforms` 가 맡으므로 여기서는 건너뜁니다.

```
피아노롤 노트 변화 → 'sq:roll-notes' { clip, notes }
└ initClipPreviews 가 듣는다
   └ setClipNotes(clip, notes)   ← 노트를 담고
      └ drawClipPreview(canvas, clip)  ← 음높이·길이대로 막대를 그림
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` 시작부	`initClipPreviews()` 로 듣기를 시작합니다
피아노롤 (224)	노트가 바뀔 때마다 이벤트를 보냅니다
S1-10 (예정)	프로젝트를 불러올 때 받은 노트를 `setClipNotes` 로 담습니다

3. 코드

```
/** MIDI 클립 → 미리보기 노트 목록 (피아노롤에서 바뀔 때마다 채운다) */
const clipNotes = new WeakMap();

export function setClipNotes(clip, notes) {
  clipNotes.set(clip, notes || []);
  drawClipPreview(clip.querySelector('.sq-clip__preview'), clip);
}

export function drawClipPreview(canvas, clip) {
  if (!canvas || !clip || !clip.classList.contains('sq-clip--midi')) return;
  const dpr = window.devicePixelRatio || 1;
  const w = canvas.clientWidth;
  const h = canvas.clientHeight;
  const pw = Math.max(1, Math.round(w * dpr));
  const ph = Math.max(1, Math.round(h * dpr));
```

```
  if (canvas.width !== pw || canvas.height !== ph) { canvas.width = pw; canvas.height = ph; }
  const ctx = canvas.getContext('2d');
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, pw, ph);
  const notes = clipNotes.get(clip) || [];
  if (w < 2 || h < 2 || !notes.length) return;
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  const bars = parseFloat(clip.style.getPropertyValue('--l')) || 1;   // 클립 길이(마디)
  const ticks = bars * TICKS_PER_BAR;
  let lo = Infinity;
  let hi = -Infinity;
  for (const n of notes) { if (n.pitch < lo) lo = n.pitch; if (n.pitch > hi) hi = n.pitch; }
  if (hi === lo) { lo -= 1; hi += 1; }
  const pad = 0.12;
  const barH = Math.max(1, Math.round(h * 0.14));
  ctx.fillStyle = getComputedStyle(clip).getPropertyValue('--tc').trim() || 'currentColor';
  for (const n of notes) {
    const x = (n.start_tick / ticks) * w;
    const bw = Math.max(1, (n.length_tick / ticks) * w);
    const y = (1 - (n.pitch - lo) / (hi - lo)) * (1 - pad * 2) + pad;
    ctx.fillRect(x, y * h - barH / 2, Math.min(bw, w - x), barH);
  }
}

function initClipPreviews() {
  document.addEventListener('sq:roll-notes', (e) => {
    const detail = e.detail || {};
    if (detail.clip) setClipNotes(detail.clip, detail.notes || []);
  });
}
initClipPreviews();
```

`sequencer.js` 맨 위 import 는 `TICKS_PER_BAR` 를 더합니다.

```
import { initArrange, TICKS_PER_BAR, restoreOpenPanels } from './sequencer-arrange.js';
```

문법 풀이

식	뜻
`new WeakMap()`	클립 요소를 열쇠로 노트 목록을 담습니다. 클립이 지워지면 함께 사라집니다
`devicePixelRatio(dpr)`	고해상도 화면에서 선명하게 그리려고 캔버스 실제 크기를 배수만큼 키웁니다
`canvas.width = pw`	캔버스 실제 픽셀 크기(그리는 좌표계)
`canvas.clientWidth`	화면에 보이는 크기(CSS 픽셀)
`ctx.clearRect(...)`	먼저 캔버스를 비웁니다
`clip.classList.contains('sq-clip--midi')`	MIDI 클립만 그립니다 (오디오는 편곡이 맡음)
`clip.style.getPropertyValue('--l')`	클립 길이(마디)를 CSS 변수에서 읽습니다
`(n.start_tick / ticks) * w`	노트의 가로 위치(픽셀)
`1 - (n.pitch - lo) / (hi - lo)`	음높이를 세로로 뒤집어 매핑(높은 음이 위)
`ctx.fillRect(...)`	노트 막대 하나를 그립니다

4. 왜 이렇게

선택	이유
클립 길이 기준으로 나눔	노트의 tick 을 클립 마디 수로 나눠 가로 위치를 정합니다. 클립을 늘리면 미리보기도 넓어집니다
화면에 보이는 음역만 사용	노트의 최저·최고 음 사이에 맞춰 그려 변화가 잘 보입니다. 한 음뿐이면 위아래로 1칸씩 벌립니다
`WeakMap` 으로 담음	별도 자료구조를 관리하지 않아도 클립이 사라지면 담긴 노트도 정리됩니다
이벤트로 받음	피아노롤이 클립을 직접 알 필요 없이 문서 이벤트(224)만 보면 됩니다
오디오는 건너뜀	오디오 클립 미리보기는 S1-06 의 `drawWaveforms` 가 파형을 그립니다. 여기서 덮으면 안 됩니다

5. 시험

헤드리스 브라우저에서 빈 캔버스(픽셀 0) → 노트 4개를 담으니 156픽셀, 이벤트로 노트 2개를 받으니 92픽셀, 빈 목록으로 지우니 다시 0픽셀이 되는 것을 확인했습니다. 피아노롤 편집(`'sq:roll-notes'`)으로 클립이 403픽셀 그려지는 것도 확인했습니다(콘솔 오류 0건). 캡처는 `docs/screenshots/2026-10-17_s1-08_fixes/07_clip_preview_midi.png` 입니다.